

Socket Programming in Java



Interprocess Communication (IPC)

- IPC allows different processes to communicate with each other
- IPC in the **Internet** provides both datagram (UDP) and stream (TCP) communication
 - Java API for IPC provides both datagram and stream communication
- IPC supports point-to-point communication and multicast communication
 - Point-to-point: one sender and one receiver
 - Multicast: one sender and a group of receivers

Point – to – Point Communication

Client-server communication

- Client: **requests** a service
- Server: **provides** the service

Sockets

- IPC consists of transmitting a message between a **socket** in one process and a socket in another process
- A socket
 - IPv4 address
 - Port number
- TCP sockets, UDP sockets

TCP Socket: Example with Echo-server project

TCP

- Connection-oriented protocol
 - Three-way handshake
- Data type: Stream

TCP sockets

- A TCP socket is treated as a file
- **Send()** operation: Write data to the file
- **Receive()** operation: Read data from the file

TCP socket implementation

- Open a socket.
- Open an input stream and output stream to the socket
- Read from and write to the stream according to the server's protocol
- Close the streams
- Close the socket

How the EchoServer project works:

- Client reads input from user
- Client sends the input to the server
- The server sends the message back to the client
- The client displays the message in the console

- Client and server can be run in the same or different computers

Server implementation

```
public class EchoServer {  
    public static void main(String[] args) throws IOException {  
  
        int portNumber = 6565;  
  
        System.out.printf("Starting Server at port %d\n", portNumber);  
  
        try (ServerSocket serverSocket = new ServerSocket(portNumber);  
            Socket clientSocket = serverSocket.accept();  
            PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);  
            BufferedReader in = new BufferedReader(new InputStreamReader(clientSocket.getInputStream()));) {  
            String inputLine;  
            while ((inputLine = in.readLine()) != null) {  
                out.println(inputLine);  
            }  
        } catch (IOException e) {  
            System.out.println(  
                "Exception caught when trying to listen on port " + portNumber + " or listening for a connection");  
            System.out.println(e.getMessage());  
        }  
    }  
}
```

Run the service on port 6565.
Why is server address not
needed?

Server implementation

```
public class EchoServer {  
    public static void main(String[] args) throws IOException {  
  
        int portNumber = 6565;  
  
        System.out.printf("Starting Server at port %d\n", portNumber);  
  
        try {ServerSocket serverSocket = new ServerSocket(portNumber);  
            Socket clientSocket = serverSocket.accept();  
            PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);  
            BufferedReader in = new BufferedReader(new InputStreamReader(clientSocket.getInputStream()));  
            String inputline;  
            while ((inputline = in.readLine()) != null) {  
                out.println(inputline);  
            }  
        } catch (IOException e) {  
            System.out.println(  
                "Exception caught when trying to listen on port " + portNumber + " or listening for a connection");  
            System.out.println(e.getMessage());  
        }  
    }  
}
```

Server is opening the port to run the service.

Server implementation

```
public class EchoServer {  
    public static void main(String[] args) throws IOException {  
  
        int portNumber = 6565;  
  
        System.out.printf("Starting Server at port %d\n", portNumber);  
  
        try (ServerSocket serverSocket = new ServerSocket(portNumber);  
            Socket clientSocket = serverSocket.accept();  
            PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);  
            BufferedReader in = new BufferedReader(new InputStreamReader(clientSocket.getInputStream()));) {  
            String inputLine;  
            while ((inputLine = in.readLine()) != null) {  
                out.println(inputLine);  
            }  
        } catch (IOException e) {  
            System.out.println(  
                "Exception caught when trying to listen on port " + portNumber + " or listening for a connection");  
            System.out.println(e.getMessage());  
        }  
    }  
}
```

Connection establishment

- Waiting for request from client
- Blocking call.

Server implementation

```
public class EchoServer {  
    public static void main(String[] args) throws IOException {  
  
        int portNumber = 6565;  
  
        System.out.printf("Starting Server at port %d\n", portNumber);  
  
        try (ServerSocket serverSocket = new ServerSocket(portNumber);  
            Socket clientSocket = serverSocket.accept();  
            PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);  
            BufferedReader in = new BufferedReader(new InputStreamReader(clientSocket.getInputStream()));) {  
            String inputLine;  
            while ((inputLine = in.readLine()) != null) {  
                out.println(inputLine);  
            }  
        } catch (IOException e) {  
            System.out.println(  
                "Exception caught when trying to listen on port " + portNumber + " or listening for a connection");  
            System.out.println(e.getMessage());  
        }  
    }  
}
```

- Open output stream and use
to write output data

Server implementation

```
public class EchoServer {  
    public static void main(String[] args) throws IOException {  
  
        int portNumber = 6565;  
  
        System.out.printf("Starting Server at port %d\n", portNumber);  
  
        try (ServerSocket serverSocket = new ServerSocket(portNumber);  
            Socket clientSocket = serverSocket.accept();  
            PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);  
            BufferedReader in = new BufferedReader(new InputStreamReader(clientSocket.getInputStream()));) {  
            String inputLine;  
            while ((inputLine = in.readLine()) != null) {  
                out.println(inputLine);  
            }  
        } catch (IOException e) {  
            System.out.println(  
                "Exception caught when trying to listen on port " + portNumber + " or listening for a connection");  
            System.out.println(e.getMessage());  
        }  
    }  
}
```

- Get input stream and read data

Server implementation

```
public class EchoServer {  
    public static void main(String[] args) throws IOException {  
  
        int portNumber = 6565;  
  
        System.out.printf("Starting Server at port %d\n", portNumber);  
  
        try (ServerSocket serverSocket = new ServerSocket(portNumber);  
            Socket clientSocket = serverSocket.accept();  
            PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);  
            BufferedReader in = new BufferedReader(new InputStreamReader(clientSocket.getInputStream()));) {  
            String inputLine;  
            while ((inputLine = in.readLine()) != null) {  
                out.println(inputLine);  
            }  
        } catch (IOException e) {  
            System.out.println(  
                "Exception caught when trying to listen on port " + portNumber + " or listening for a connection");  
            System.out.println(e.getMessage());  
        }  
    }  
}
```

Server reads input from
the client.

Client implementation

```
public class EchoClient {
    public static void main(String[] args) throws IOException {

        String hostName = "localhost"; //127.0.0.1, 192.168.6.103
        String hostName = "192.168.6.103";
        int portNumber = 6565;

        try (Socket echoSocket = new Socket(hostName, portNumber);
            PrintWriter out = new PrintWriter(echoSocket.getOutputStream(), true);
            BufferedReader in = new BufferedReader(new InputStreamReader(echoSocket.getInputStream()));
            BufferedReader stdIn = new BufferedReader(new InputStreamReader(System.in))) {
            String userInput;
            while ((userInput = stdIn.readLine()) != null) {

                if (userInput.equals("exit"))
                    break;

                out.println(userInput);
                System.out.println("echo: " + in.readLine());
            }
        } catch (UnknownHostException e) {
            System.err.println("Don't know about host " + hostName);
            System.exit(1);
        } catch (IOException e) {
            System.err.println("Couldn't get I/O for the connection to " + hostName);
            System.exit(1);
        }
    }
}
```

- Server IP's address if run on the same computer.
- Otherwise, use the Server IPv4 address

Client implementation

```
public class EchoClient {  
    public static void main(String[] args) throws IOException {  
  
        String hostName = "localhost"; //127.0.0.1, 192.168.6.103  
        // String hostName = "192.168.6.103";  
        int portNumber = 6565;  
  
        try (Socket echoSocket = new Socket(hostName, portNumber);  
            PrintWriter out = new PrintWriter(echoSocket.getOutputStream(), true);  
            BufferedReader in = new BufferedReader(new InputStreamReader(echoSocket.getInputStream()));  
            BufferedReader stdIn = new BufferedReader(new InputStreamReader(System.in))) {  
            String userInput;  
            while ((userInput = stdIn.readLine()) != null) {  
  
                if (userInput.equals("exit"))  
                    break;  
  
                out.println(userInput);  
                System.out.println("echo: " + in.readLine());  
            }  
        } catch (UnknownHostException e) {  
            System.err.println("Don't know about host " + hostName);  
            System.exit(1);  
        } catch (IOException e) {  
            System.err.println("Couldn't get I/O for the connection to " + hostName);  
            System.exit(1);  
        }  
    }  
}
```

Port number: the port that the server is running the service.

Client implementation

```
public class EchoClient {  
    public static void main(String[] args) throws IOException {  
  
        String hostName = "localhost"; //127.0.0.1, 192.168.6.103  
        // String hostName = "192.168.6.103";  
        int portNumber = 6565;  
  
        try (Socket echoSocket = new Socket(hostName, portNumber);  
            PrintWriter out = new PrintWriter(echoSocket.getOutputStream(), true);  
            BufferedReader in = new BufferedReader(new InputStreamReader(echoSocket.getInputStream()));  
            BufferedReader stdIn = new BufferedReader(new InputStreamReader(System.in))) {  
            String userInput;  
            while ((userInput = stdIn.readLine()) != null) {  
  
                if (userInput.equals("exit"))  
                    break;  
  
                out.println(userInput);  
                System.out.println("echo: " + in.readLine());  
            }  
        } catch (UnknownHostException e) {  
            System.err.println("Don't know about host " + hostName);  
            System.exit(1);  
        } catch (IOException e) {  
            System.err.println("Couldn't get I/O for the connection to " + hostName);  
            System.exit(1);  
        }  
    }  
}
```

Port number: service that the server offers.

Client implementation

```
public class EchoClient {  
    public static void main(String[] args) throws IOException {
```

```
        String hostName = "localhost"; //127.0.0.1, 192.168.6.103  
        // String hostName = "192.168.6.103";  
        int portNumber = 6565;
```

Creates a socket to connect to the server.

```
        try {  
            (Socket echoSocket = new Socket(hostName, portNumber);  
            PrintWriter out = new PrintWriter(echoSocket.getOutputStream(), true);  
            BufferedReader in = new BufferedReader(new InputStreamReader(echoSocket.getInputStream()));  
            BufferedReader stdIn = new BufferedReader(new InputStreamReader(System.in)) {  
                String userInput;  
                while ((userInput = stdIn.readLine()) != null) {
```

```
                    if (userInput.equals("exit"))  
                        break;
```

```
                    out.println(userInput);  
                    System.out.println("echo: " + in.readLine());
```

```
                }  
            } catch (UnknownHostException e) {  
                System.err.println("Don't know about host " + hostName);  
                System.exit(1);  
            } catch (IOException e) {  
                System.err.println("Couldn't get I/O for the connection to " + hostName);  
                System.exit(1);  
            }  
        }  
    }
```

```
}
```

Client implementation

```
public class EchoClient {  
    public static void main(String[] args) throws IOException {  
  
        String hostName = "localhost"; //127.0.0.1, 192.168.6.103  
        // String hostName = "192.168.6.103";  
        int portNumber = 6565;  
  
        try (Socket echoSocket = new Socket(hostName, portNumber);  
            PrintWriter out = new PrintWriter(echoSocket.getOutputStream(), true);  
            BufferedReader in = new BufferedReader(new InputStreamReader(echoSocket.getInputStream()));  
            BufferedReader stdIn = new BufferedReader(new InputStreamReader(System.in))) {  
            String userInput;  
            while ((userInput = stdIn.readLine()) != null) {  
  
                if (userInput.equals("exit"))  
                    break;  
  
                out.println(userInput);  
                System.out.println("echo: " + in.readLine());  
            }  
        } catch (UnknownHostException e) {  
            System.err.println("Don't know about host " + hostName);  
            System.exit(1);  
        } catch (IOException e) {  
            System.err.println("Couldn't get I/O for the connection to " + hostName);  
            System.exit(1);  
        }  
    }  
}
```

- Get the socket's output Stream.

Client implementation

```
public class EchoClient {  
    public static void main(String[] args) throws IOException {  
  
        String hostName = "localhost"; //127.0.0.1, 192.168.6.103  
        // String hostName = "192.168.6.103";  
        int portNumber = 6565;  
  
        try (Socket echoSocket = new Socket(hostName, portNumber);  
            PrintWriter out = new PrintWriter(echoSocket.getOutputStream(), true);  
            BufferedReader in = new BufferedReader(new InputStreamReader(echoSocket.getInputStream()));  
            BufferedReader stdin = new BufferedReader(new InputStreamReader(System.in)); {  
            String userInput;  
            while ((userInput = stdin.readLine()) != null) {  
  
                if (userInput.equals("exit"))  
                    break;  
  
                out.println(userInput);  
                System.out.println("echo: " + in.readLine());  
            }  
        } catch (UnknownHostException e) {  
            System.err.println("Don't know about host " + hostName);  
            System.exit(1);  
        } catch (IOException e) {  
            System.err.println("Couldn't get I/O for the connection to " + hostName);  
            System.exit(1);  
        }  
    }  
}
```

Get the input Stream.

Client implementation

```
public class EchoClient {
    public static void main(String[] args) throws IOException {

        String hostName = "localhost"; //127.0.0.1, 192.168.6.103
        // String hostName = "192.168.6.103";
        int portNumber = 6565;

        try (Socket echoSocket = new Socket(hostName, portNumber);
            PrintWriter out = new PrintWriter(echoSocket.getOutputStream(), true);
            BufferedReader in = new BufferedReader(new InputStreamReader(echoSocket.getInputStream()));
            BufferedReader stdIn = new BufferedReader(new InputStreamReader(System.in))) {
            String userInput;
            while ((userInput = stdIn.readLine()) != null) {

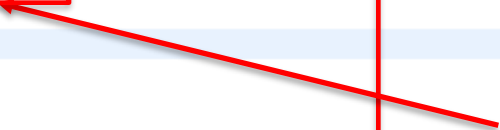
                if (userInput.equals("exit"))
                    break;

                out.println(userInput);
                System.out.println("echo: " + in.readLine());
            }
        } catch (UnknownHostException e) {
            System.err.println("Don't know about host " + hostName);
            System.exit(1);
        } catch (IOException e) {
            System.err.println("Couldn't get I/O for the connection to " + hostName);
            System.exit(1);
        }
    }
}
```

Read input from user console.

Client implementation

```
public class EchoClient {  
    public static void main(String[] args) throws IOException {  
  
        String hostName = "localhost"; //127.0.0.1, 192.168.6.103  
        // String hostName = "192.168.6.103";  
        int portNumber = 6565;  
  
        try (Socket echoSocket = new Socket(hostName, portNumber);  
            PrintWriter out = new PrintWriter(echoSocket.getOutputStream(), true);  
            BufferedReader in = new BufferedReader(new InputStreamReader(echoSocket.getInputStream()));  
            BufferedReader stdIn = new BufferedReader(new InputStreamReader(System.in))) {  
            String userInput;  
            while ((userInput = stdIn.readLine()) != null) {  
                if (userInput.equals("exit"))  
                    break;  
  
                out.println(userInput);  
                System.out.println("echo: " + in.readLine());  
            }  
        } catch (UnknownHostException e) {  
            System.err.println("Don't know about host " + hostName);  
            System.exit(1);  
        } catch (IOException e) {  
            System.err.println("Couldn't get I/O for the connection to " + hostName);  
            System.exit(1);  
        }  
    }  
}
```



User Input

Client implementation

```
public class EchoClient {  
    public static void main(String[] args) throws IOException {  
  
        String hostName = "localhost"; //127.0.0.1, 192.168.6.103  
        // String hostName = "192.168.6.103";  
        int portNumber = 6565;  
  
        try (Socket echoSocket = new Socket(hostName, portNumber);  
            PrintWriter out = new PrintWriter(echoSocket.getOutputStream(), true);  
            BufferedReader in = new BufferedReader(new InputStreamReader(echoSocket.getInputStream()));  
            BufferedReader stdIn = new BufferedReader(new InputStreamReader(System.in))) {  
            String userInput;  
            while ((userInput = stdIn.readLine()) != null) {  
                if (userInput.equals("exit"))  
                    break;  
  
                out.println(userInput);  
                System.out.println("echo: " + in.readLine());  
            }  
        } catch (UnknownHostException e) {  
            System.err.println("Don't know about host " + hostName);  
            System.exit(1);  
        } catch (IOException e) {  
            System.err.println("Couldn't get I/O for the connection to " + hostName);  
            System.exit(1);  
        }  
    }  
}
```

Send data (output stream) to the server.

Client implementation

```
public class EchoClient {  
    public static void main(String[] args) throws IOException {  
  
        String hostName = "localhost"; //127.0.0.1, 192.168.6.103  
        // String hostName = "192.168.6.103";  
        int portNumber = 6565;  
  
        try (Socket echoSocket = new Socket(hostName, portNumber);  
            PrintWriter out = new PrintWriter(echoSocket.getOutputStream(), true);  
            BufferedReader in = new BufferedReader(new InputStreamReader(echoSocket.getInputStream()));  
            BufferedReader stdIn = new BufferedReader(new InputStreamReader(System.in))) {  
            String userInput;  
            while ((userInput = stdIn.readLine()) != null) {  
                if (userInput.equals("exit"))  
                    break;  
  
                out.println(userInput);  
                System.out.println("echo: " + in.readLine());  
            }  
        } catch (UnknownHostException e) {  
            System.err.println("Don't know about host " + hostName);  
            System.exit(1);  
        } catch (IOException e) {  
            System.err.println("Couldn't get I/O for the connection to " + hostName);  
            System.exit(1);  
        }  
    }  
}
```

Receive message from server.

Multithread server

- When a server has to handle multiple requests from multiple clients
 - Each request can be considered as a separated task.
 - We can use multithread programming to implement server
- Should we create one thread for each incoming tasks or should we use a thread pool to handle all tasks?

UDP Sockets



UDP sockets

- UDP
 - Connectionless protocol
 - No connection establishment
 - Datagram

- Using send() for send operation
- Using receive() for receive operation

```
int server_port = 9802;
DatagramSocket serverSocket = new DatagramSocket(server_port);
System.out.println("Waiting for client on port " + server_port);
byte[] receiveData = new byte[1024];
while (true) {
    DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);
    serverSocket.receive(receivePacket);
    String sentence = new String(receivePacket.getData());
    InetAddress IPAddress = receivePacket.getAddress();
    int client_port = receivePacket.getPort();

    System.out.println("Received from client: " + sentence);
    System.out.println("Message sent from IP address: " + IPAddress + " and port: " + client_port);

    sentence = "Welcome from server" ;
    byte[] sendData = sentence.getBytes();
    DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length, IPAddress, client_port);
    serverSocket.send(sendPacket);
}
```

Define port service

```
int server_port = 9802;
DatagramSocket serverSocket = new DatagramSocket(server_port);
System.out.println("Waiting for client on port " + server_port);
byte[] receiveData = new byte[1024];
while (true) {
    DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);
    serverSocket.receive(receivePacket);
    String sentence = new String(receivePacket.getData());
    InetAddress IPAddress = receivePacket.getAddress();
    int client_port = receivePacket.getPort();

    System.out.println("Received from client: " + sentence);
    System.out.println("Message sent from IP address: " + IPAddress + " and port: " + client_port);

    sentence = "Welcome from server" ;
    byte[] sendData = sentence.getBytes();
    DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length, IPAddress, client_port);
    serverSocket.send(sendPacket);
}
```

Create a socket using
DatagramSocket class

```
int server_port = 9802;
DatagramSocket serverSocket = new DatagramSocket(server_port);
System.out.println("Waiting for client on port " + server_port);
byte[] receiveData = new byte[1024];
while (true) {
    DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);
    serverSocket.receive(receivePacket);
    String sentence = new String(receivePacket.getData());
    InetAddress IPAddress = receivePacket.getAddress();
    int client_port = receivePacket.getPort();

    System.out.println("Received from client: " + sentence);
    System.out.println("Message sent from IP address: " + IPAddress + " and port: " + client_port);

    sentence = "Welcome from server" ;
    byte[] sendData = sentence.getBytes();
    DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length, IPAddress, client_port);
    serverSocket.send(sendPacket);
}
```

Create an array and prepare to receive data from client


```
int server_port = 9802;
DatagramSocket serverSocket = new DatagramSocket(server_port);
System.out.println("Waiting for client on port " + server_port);
byte[] receiveData = new byte[1024];
while (true) {
    DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);
    serverSocket.receive(receivePacket);
    String sentence = new String(receivePacket.getData());
    InetAddress IPAddress = receivePacket.getAddress();
    int client_port = receivePacket.getPort();

    System.out.println("Received from client: " + sentence);
    System.out.println("Message sent from IP address: " + IPAddress + " and port: " + client_port);

    sentence = "Welcome from server" ;
    byte[] sendData = sentence.getBytes();
    DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length, IPAddress, client_port);
    serverSocket.send(sendPacket);
}
```

Receive data from client using
receive operation

```
int server_port = 9802;
DatagramSocket serverSocket = new DatagramSocket(server_port);
System.out.println("Waiting for client on port " + server_port);
byte[] receiveData = new byte[1024];
while (true) {
    DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);
    serverSocket.receive(receivePacket);
    String sentence = new String(receivePacket.getData());
    InetAddress IPAddress = receivePacket.getAddress();
    int client_port = receivePacket.getPort();

    System.out.println("Received from client: " + sentence);
    System.out.println("Message sent from IP address: " + IPAddress + " and port: " + client_port);

    sentence = "Welcome from server" ;
    byte[] sendData = sentence.getBytes();
    DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length, IPAddress, client_port);
    serverSocket.send(sendPacket);
}
```

Send data to client using send() operation

```
int server_port = 9802;
DatagramSocket serverSocket = new DatagramSocket(server_port);
System.out.println("Waiting for client on port " + server_port);
byte[] receiveData = new byte[1024];
while (true) {
    DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);
    serverSocket.receive(receivePacket);

    String sentence = new String(receivePacket.getData());
    InetAddress IPAddress = receivePacket.getAddress();
    int client_port = receivePacket.getPort();

    System.out.println("Received from client: " + sentence);
    System.out.println("Message sent from IP address: " + IPAddress + " and port: " + client_port);

    sentence = "Welcome from server" ;

    byte[] sendData = sentence.getBytes();
    DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length, IPAddress, client_port);
    serverSocket.send(sendPacket);
}
```

Different for different service.

```
int server_port = 9802;
DatagramSocket serverSocket = new DatagramSocket(server_port);
System.out.println("Waiting for client on port " + server_port);
byte[] receiveData = new byte[1024];
while (true) {
    DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);
    serverSocket.receive(receivePacket);
    String sentence = new String(receivePacket.getData());
    InetAddress IPAddress = receivePacket.getAddress();
    int client_port = receivePacket.getPort();

    System.out.println("Received from client: " + sentence);
    System.out.println("Message sent from IP address: " + IPAddress + " and port: " + client_port);
```

Make DatagramPacket object to be sent or received.

```
sentence = "Welcome from server" ;
byte[] sendData = sentence.getBytes();
DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length, IPAddress, client_port);
serverSocket.send(sendPacket);
```

try {

`DatagramSocket clientSocket = new DatagramSocket();``InetAddress IPAddress = InetAddress.getByName("localhost");``int port = 9802;``String sentence = "Hello world";``byte[] sendData = sentence.getBytes();``DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length, IPAddress, port);``clientSocket.send(sendPacket);``byte[] receiveData = new byte[1024];``DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);``clientSocket.receive(receivePacket);``String receivedSentence = new String(receivePacket.getData());``System.out.println("Received from server: " + receivedSentence);``clientSocket.close();``} catch (Exception e) {` `e.printStackTrace();``}`

- Create a socket with server's IP address and port number in which the service is offered.
- Using DatagramSocket

Running program with multiple clients

Server port

```
Waiting for client on port 9802
Received from client: Hello world
Message sent from IP address: /127.0.0.1 and port: 51106
Received from client: Hello world
Message sent from IP address: /127.0.0.1 and port: 61165
Received from client: Hello world
Message sent from IP address: /127.0.0.1 and port: 61166
Received from client: Hello world
Message sent from IP address: /127.0.0.1 and port: 54959
```

Client ports

Why multiple client ports?

Multicast Communication

IP Multicast – An Implementation of multicast communication

- Built on the top of Internet Protocol (IP)
- IP multicasts allows a sender to send a single IP packet to a set of computers that form a multicast group
 - IP packets: addressed to computers
 - Ports: TCP, UDP levels

Key features

- Multicast is based on a **group membership** concept
 - Receivers have to join or subscribe to receive a packet sent to group
- The sender is not aware of each member in the group
- The sender does not need to be a member of the group
- Based on UDP protocol

Class D Internet Address

	octet 1	octet 2	octet 3		Range of addresses
	Network ID		Host ID		
Class A:	1 to 127	0 to 255	0 to 255	0 to 255	1.0.0.0 to 127.255.255.255
	Network ID		Host ID		
Class B:	128 to 191	0 to 255	0 to 255	0 to 255	128.0.0.0 to 191.255.255.255
	Network ID			Host ID	
Class C:	192 to 223	0 to 255	0 to 255	1 to 254	192.0.0.0 to 223.255.255.255
	Multicast address				
Class D (multicast):	224 to 239	0 to 255	0 to 255	1 to 254	224.0.0.0 to 239.255.255.255
Class E (reserved):	240 to 255	0 to 255	0 to 255	1 to 254	240.0.0.0 to 255.255.255.255

Implementation

- Use multicast address
- Use **MulticastSocket** class which is a subclass of **DatagramSocket** class
- Use **UDP** protocol

Receiver Implementation

```
String multicastAddress = "224.0.0.3";
```

```
int port = 6565;
```

- Multicast group and port
- A multicast group is identified by an IP address
- All receivers use the same port number to listen for incoming traffic

```
try (MulticastSocket socket = new MulticastSocket(port)) {  
    InetAddress group = InetAddress.getByName(multicastAddress);  
    socket.joinGroup(group); // Join (subscribe) the multicast group  
    System.out.println("Receiver listening for multicast messages...");  
  
    byte[] buffer = new byte[256];  
    DatagramPacket packet = new DatagramPacket(buffer, buffer.length);  
  
    socket.receive(packet); // Wait for incoming message  
    String receivedMessage = new String(packet.getData(), 0, packet.getLength());  
    System.out.println("Receiver received: " + receivedMessage);  
  
    socket.leaveGroup(group); // Leave group after receiving
```

Receiver Implementation

```
String multicastAddress = "224.0.0.3";
```

```
int port = 6565;
```

```
try (MulticastSocket socket = new MulticastSocket(port)) {
```

Create a multicast socket to
receive a packet

```
    InetAddress group = InetAddress.getByName(multicastAddress);
```

```
    socket.joinGroup(group); // Join (subscribe) the multicast group
```

```
    System.out.println("Receiver listening for multicast messages...");
```

```
    byte[] buffer = new byte[256];
```

```
    DatagramPacket packet = new DatagramPacket(buffer, buffer.length);
```

```
    socket.receive(packet); // Wait for incoming message
```

```
    String receivedMessage = new String(packet.getData(), 0, packet.getLength());
```

```
    System.out.println("Receiver received: " + receivedMessage);
```

```
    socket.leaveGroup(group); // Leave group after receiving
```

Receiver Implementation

```
String multicastAddress = "224.0.0.3";  
int port = 6565;  
  
try (MulticastSocket socket = new MulticastSocket(port)) {  
    InetAddress group = InetAddress.getByName(multicastAddress);  
    socket.joinGroup(group); // Join (subscribe) the multicast group  
    System.out.println("Receiver listening for multicast messages...");  
  
    byte[] buffer = new byte[256];  
    DatagramPacket packet = new DatagramPacket(buffer, buffer.length);  
  
    socket.receive(packet); // Wait for incoming message  
    String receivedMessage = new String(packet.getData(), 0, packet.getLength());  
    System.out.println("Receiver received: " + receivedMessage);  
  
    socket.leaveGroup(group); // Leave group after receiving
```

A receiver has to join the multicast group to receive a packet sent to the group

Receiver Implementation

```
String multicastAddress = "224.0.0.3";  
int port = 6565;  
  
try (MulticastSocket socket = new MulticastSocket(port)) {  
    InetAddress group = InetAddress.getByName(multicastAddress);  
    socket.joinGroup(group); // Join (subscribe) the multicast group  
    System.out.println("Receiver listening for multicast messages...");  
  
    byte[] buffer = new byte[256];  
    DatagramPacket packet = new DatagramPacket(buffer, buffer.length);  
  
    socket.receive(packet); // Wait for incoming message  
    String receivedMessage = new String(packet.getData(), 0, packet.getLength());  
    System.out.println("Receiver received: " + receivedMessage);  
  
    socket.leaveGroup(group); // Leave group after receiving
```

Create a datagram
(UDP) packet to
receive the packet

Receiver Implementation

```
String multicastAddress = "224.0.0.3";
```

```
int port = 6565;
```

```
try (MulticastSocket socket = new MulticastSocket(port)) {  
    InetAddress group = InetAddress.getByAddress(multicastAddress);  
    socket.joinGroup(group); // Join (subscribe) the multicast group  
    System.out.println("Receiver listening for multicast messages...");
```

```
byte[] buffer = new byte[256];
```

```
DatagramPacket packet = new DatagramPacket(buffer, buffer.length);
```

```
socket.receive(packet); // Wait for incoming message
```

Receive the packet

```
String receivedMessage = new String(packet.getData(), 0, packet.getLength());
```

```
System.out.println("Receiver received: " + receivedMessage);
```

```
socket.leaveGroup(group); // Leave group after receiving
```


Receiver Implementation

```
String multicastAddress = "224.0.0.3";  
int port = 6565;  
  
try (MulticastSocket socket = new MulticastSocket(port)) {  
    InetAddress group = InetAddress.getByName(multicastAddress);  
    socket.joinGroup(group); // Join (subscribe) the multicast group  
    System.out.println("Receiver listening for multicast messages...");  
  
    byte[] buffer = new byte[256];  
    DatagramPacket packet = new DatagramPacket(buffer, buffer.length);  
  
    socket.receive(packet); // Wait for incoming message  
    String receivedMessage = new String(packet.getData(), 0, packet.getLength());  
    System.out.println("Receiver received: " + receivedMessage);  
  
    socket.leaveGroup(group); // Leave group after receiving
```

Leave group

Sender Implementation

```
String multicastAddress = "224.0.0.3"; // Multicast IP address
int port = 6565; // Port number
try (DatagramSocket socket = new DatagramSocket()) {
    InetAddress group = InetAddress.getByName(multicastAddress);
    String message = "Hello, This is a multicast message!";
    byte[] buffer = message.getBytes();

    DatagramPacket packet = new DatagramPacket(buffer, buffer.length, group, port);
    socket.send(packet);
    System.out.println("Message sent to multicast group: " + multicastAddress);
}
```

Multicast IP address
and destination port
number

Sender Implementation

```
String multicastAddress = "224.0.0.3"; // Multicast IP address
```

```
int port = 6565; // Port number
```

```
try (DatagramSocket socket = new DatagramSocket()) {
```

```
    InetAddress group = InetAddress.getByAddress(multicastAddress);
```

```
    String message = "Hello, This is a multicast message!";
```

```
    byte[] buffer = message.getBytes();
```

```
DatagramPacket packet = new DatagramPacket(buffer, buffer.length, group, port);
```

```
socket.send(packet);
```

```
System.out.println("Message sent to multicast group: " + multicastAddress);
```

Create a datagram socket to
send a packet

Sender Implementation

```
String multicastAddress = "224.0.0.3"; // Multicast IP address
int port = 6565; // Port number
try (DatagramSocket socket = new DatagramSocket()) {
    InetAddress group = InetAddress.getByAddress(multicastAddress);
    String message = "Hello, This is a multicast message!";
    byte[] buffer = message.getBytes();
```

Create a datagram packet

```
DatagramPacket packet = new DatagramPacket(buffer, buffer.length, group, port);
socket.send(packet);
System.out.println("Message sent to multicast group: " + multicastAddress);
```

Sender Implementation

```
String multicastAddress = "224.0.0.3"; // Multicast IP address
int port = 6565; // Port number
try (DatagramSocket socket = new DatagramSocket()) {
    InetAddress group = InetAddress.getByName(multicastAddress);
    String message = "Hello, This is a multicast message!";
    byte[] buffer = message.getBytes();

    DatagramPacket packet = new DatagramPacket(buffer, buffer.length, group, port);
    socket.send(packet);
    System.out.println("Message sent to multicast group: " + multicastAddress);
}
```

Send the packet